Brief Summary of Week 5

Objective- Learn the basics of Apache Spark and use it to clean, transform, and analyse data using DataFrames.

What I Learn?

- What Spark is and why it is faster than MapReduce?
- How to use Spark DataFrames?
- How to clean and process data?
- How to perform basic analysing using Spark?

What is Apache Spark and Why is it Faster than MapReduce?

Apache Spark is an open-source big data processing framework used for large-scale data processing and analytics.

What is MapReduce?

MapReduce is a programming model developed by Google and used in Hadoop for processing large datasets across a distributed cluster. It processes data in two phases — Map (transform) and Reduce (aggregate). However, it has significant limitations:

- Every intermediate result is written to HDFS disk, making it extremely slow.
- Not suitable for real-time or interactive data processing.
- Complex Code
- Slow for Iterative Jobs

Why Apache Spark is Better ?

Apache Spark overcomes all major limitations of MapReduce by processing data in memory (RAM) instead of disk. Key advantages are:

Feature	MapReduce	Apache Spark
Processing	Disk-based (HDFS)	In-Memory (RAM)
Speed	Slow (disk I/O)	Up to 100x faster
Ease of Use	Complex Java code	Simple Python/Scala API
Operations	Only Map + Reduce	Filter, Join, GroupBy, SQL, ML
Real-Time	Not supported	Supported (Streaming)
Fault Tolerance	Via HDFS replication	Via RDD lineage / DAG

Steps to Follow

Step 1 — Spark Setup and Session Creation

Installing PySpark

PySpark was installed using pip. The command used is-

Command	!pip install pyspark

Creating the SparkSession

A SparkSession is the single entry point for all Spark functionality. It was created with the application name 'Data':

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

spark = SparkSession.builder.appName("Data").getOrCreate()
```

The SparkSession was created successfully and all required functions were imported. This session manages the entire lifecycle of the Spark application.

Step 2 — Load and Explore Dataset

Loading the CSV File

```
df = spark.read.csv("data.csv", header=True, inferSchema=True)
```

The dataset was loaded from data.csv with header=True (first row as column names) and inferSchema=True (auto-detect data types)

Explore the data by viewing top 20 rows, counting number of rows and columns, by identifying names of columns and its data types.

Observation: The dataset contained duplicate rows, null values in 'status', 'email', 'price', and 'sale_amount' columns, and the 'raw_timestamp' column was stored as a string requiring type conversion.

Step 3 — Data Cleaning

Removing Duplicate Rows

Duplicate rows were observed while viewing the first 20 records. For example, USR_1007 on 2026-06-13 appeared twice, and USR_1047 on 2026-06-15 appeared twice. The dropDuplicates() function was used as part of the pipeline to remove them:

```
.dropDuplicates(["user_id", "transaction_date"])
```

This removed all rows where the same user had the same transaction date, keeping only one record per user per day.

Handling Missing (Null) Values

Several columns contained null or None values. The null values in the 'status' column were filled using na.fill():

```
.na.fill({"status": "Unknown"})
```

This replaced all None values in the status column with the default value 'Unknown', ensuring no missing data in critical fields.

Step 4 — Filtering Data

Filter by Age, Category, and Region

Filters were applied to extract records we can apply filters like of age, category and region.

```
filter = df.filter(
    (col("age") >= 18) &
    (col("age") <= 30) &
    (col("product_category") == "Electronics") &
    (col("region") == "West")
)
```

Step 5— Data Transformation

Type Casting: String to Timestamp and Renaming the Column

The 'raw_timestamp' column was cast from String type to proper TimestampType and renamed to 'event_time':

```
.withColumn("event_time", col("raw_timestamp").cast(TimestampType()))
.drop("raw_timestamp")
```

This is an important transformation because string timestamps cannot be used for time-based filtering or sorting. After casting, the column was renamed for clarity and the original column was dropped.

Derived Column: Age Group

A new column 'age_group' was created using the when() / otherwise() function to categorize users by age:

```
.withColumn("age_group",
    when(col("age") < 30, "Young")
    .when(col("age") < 50, "Adult")
    .otherwise("Senior"))
```

Condition	Age Group Label
age < 30	Young
30 <= age < 50	Adult
age >= 50	Senior

This derived column was used in groupBy analysis to understand purchasing behavior across age groups.

Step 6 — Aggregation Functions

Overall Sales Statistics

The following aggregation was performed on the entire dataset to get key business metrics:

```
df.agg(
    count("*").alias("total_rows"),
    sum("sale_amount").alias("total_sales"),
    avg("sale_amount").alias("average_sales"),
    min("sale_amount").alias("minimum_sales"),
    max("sale_amount").alias("maximum_sales")
).show()
```

Step 7 — Group Data

GroupBy Product Category

The groupBy() function was used to group data by product_category and calculate aggregate statistics per category. This is a wide transformation that causes a shuffle operation across partitions.

GroupBy Region

Similar groupBy was performed on the region column to understand sales performance across geographic regions. The operations applied include count(), sum(), and avg().

GroupBy Age Group

After creating the derived 'age_group' column, groupBy was applied to understand the buying behavior of Young, Adult, and Senior customers.

These groupBy operations are examples of wide transformations because data from multiple partitions must be shuffled together and reorganized by the grouping key before aggregation can be performed.

Key Insight	Young users (age < 30) tend to buy Electronics and Clothing, while Adult and Senior users lean toward Home, Beauty, and Books categories.

Step 8 — Wide Transformations & Shuffle

Narrow vs Wide Transformations

Type	Examples	Shuffle?	Performance
Narrow	filter(), select(), withColumn(), map()	NO	Fast — data stays in partition
Wide	groupBy(), join(), distinct(), orderBy()	YES	Slower — data moves across partitions

How Shuffle Works?

When a wide transformation like groupBy() is executed:

- Spark divides the data into partitions across worker nodes.
- To group rows by a key, rows with the same key may be in different partitions.
- Spark performs a SHUFFLE — moving rows across the network to the correct partition.
- This network data movement is the main cause of slowness in wide transformations.

Step 9— Building Pipeline

A **data pipeline** is a sequence of data processing steps where the output of one step becomes the input of the next step. It helps automate data cleaning, transformation, and analysis in an organized manner.

In Apache Spark, pipelines are used to process data efficiently by chaining multiple operations together.

STEP 1 LOAD DATASET

STEP 2 CLEAN DATA

STEP 3 FILTER DATA

STEP 4 APPLY TRANSFORNATIONS

STEP 5 PERFORM AGGREGATIONS

Observations-

- We don't see any changes in new updated csv by applying filters because they are only for analysis,
- We can't see any modification in updated csv by applying aggregations because they are only single-value reports.
- We can't see any modification in updated csv by applying groupBy because they are only summary tables.
- So, we are applying pipeline containing -

- STEP 1 Load dataset
- STEP 2 CLEAN DATA
- STEP 4 APPLY TRANSFORNATIONS